

XIX. Allez plus loin — Variables, animations, bonnes pratiques

LA SUITE EN UNE PHRASE

Les bases CSS sont posées. Cette leçon balaie les sujets *niveau 2* qui font la différence entre un CSS qui fonctionne et un CSS professionnel : variables, animations, BEM, perf, et les pièges à éviter.

Chapitre hors-trame séance

Ce chapitre **n'est pas traité en séance** (la trame J3 le saute pour cibler 6h30 actives et garder le TP sommatif respiré). Il est conçu pour la **lecture autonome post-cours**, quand vous voulez passer du CSS *qui marche* au CSS *professionnel*. À lire avant votre prochain projet personnel — vous gagnerez 2 heures par jour sur les charts graphiques.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Les variables CSS**
 - 1. Déclaration et usage
 - 2. Avantages par rapport à du Sass/Less
 - 3. Cas d'usage puissant : theming + mode sombre
 - 4. Avec calc()
- **B. Transitions et animations**
 - 1. Transitions — animer un changement d'état
 - 2. Animations — keyframes
 - 3. Respecter prefers-reduced-motion
- **C. Méthodologie BEM**
 - 1. Le pattern
 - 2. Exemple concret
 - 3. Pourquoi BEM ?
 - 4. Autres conventions
- **D. Performance CSS**
 - 1. Minification
 - 2. Sélecteurs simples vs complexes
 - 3. Animations GPU-friendly
 - 4. Chargement critique
 - 5. Checklist perf CSS
- **E. Préprocesseurs et frameworks — panorama**
 - 1. Préprocesseurs (Sass / SCSS)
 - 2. Frameworks CSS
 - 3. Post-processeurs (PostCSS + Autoprefixer)
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Utiliser** les variables CSS natives (`--ma-couleur`) pour centraliser les valeurs réutilisables.
- **Créer** des transitions et des animations avec `transition` et `@keyframes` .
- **Appliquer** la méthodologie **BEM** pour nommer ses classes proprement.
- **Identifier** les propriétés qui déclenchent reflow/repaint et celles qui sont GPU-friendly.
- **Lister** les bonnes pratiques de performance (minification, chargement critique, sélecteurs simples).
- **Connaître** l'existence des préprocesseurs (Sass) et des frameworks (Tailwind, Bootstrap) sans s'y enfermer.

2. En pratique — situation professionnelle

Vous reprenez un projet CSS écrit il y a 2 ans. La charte graphique change : la couleur de la marque passe de bleu à vert. Sans variables CSS, vous allez faire 40 recherches/remplacements dans 12 fichiers. Avec 1 variable `--color-brand` , c'est une seule ligne à changer.

3. Quand l'utiliser, quand l'éviter

✅ Utiliser pour	❌ Éviter quand
Variables CSS pour la charte graphique (theming, mode sombre)	Animer <code>width</code> , <code>height</code> , <code>top</code> (déclenche layout coûteux)
Animer <code>transform</code> / <code>opacity</code> (GPU-friendly, 60 fps mobile)	Empiler des sélecteurs > 3 niveaux (perf + maintenabilité)
BEM dans un projet à plusieurs développeurs	Sass en 2026 quand variables CSS natives suffisent

A. Les variables CSS

Aussi appelées **custom properties**. Natives depuis 2016, supportées partout en 2026.

1. Déclaration et usage

```
:root {  
  --color-brand: #007bff;  
  --color-text: #222;  
  --spacing-base: 1rem;  
  --radius: 6px;  
  --font-base: system-ui, sans-serif;  
}  
  
.btn {  
  background: var(--color-brand);  
  color: white;  
  padding: var(--spacing-base) calc(var(--spacing-base) * 2);  
  border-radius: var(--radius);  
  font-family: var(--font-base);  
}
```

Convention : définir les variables globales dans `:root` (équivalent de `<html>`) pour qu'elles soient disponibles partout.

2. Avantages par rapport à du Sass/Less

- **Natives** — aucun outil de build nécessaire.
- **Dynamiques** — peuvent être modifiées en live par JavaScript ou par media query.
- **Héritées** — suivent la cascade CSS comme toute propriété.

3. Cas d'usage puissant : theming + mode sombre

```
:root {
  --bg: white;
  --text: #222;
  --accent: #007bff;
}

[data-theme="dark"] {
  --bg: #111;
  --text: #eee;
  --accent: #4dabf7;
}

body {
  background: var(--bg);
  color: var(--text);
}
```

```
// basculer le thème en JS
document.documentElement.dataset.theme = 'dark';
```

→ Toutes les couleurs du site se mettent à jour **instantanément**.

4. Avec `calc()`

```
:root {
  --base: 1rem;
}

.grand { padding: calc(var(--base) * 2); } /* 2rem */
.petit { padding: calc(var(--base) / 2); } /* 0.5rem */
```

B. Transitions et animations

1. Transitions — animer un changement d'état

Pour animer une propriété qui change (au hover, au focus, par JS) :

```
.btn {  
  background: #007bff;  
  transform: scale(1);  
  transition: background 0.3s ease, transform 0.2s ease;  
}  
  
.btn:hover {  
  background: #0056b3;  
  transform: scale(1.05);  
}
```

Syntaxe `transition: propriété durée fonction délai` :

- **Propriété** — laquelle animer (`all` = toutes, mais moins performant).
- **Durée** — `0.3s` , `300ms` .
- **Fonction** — `linear` , `ease` , `ease-in` , `ease-out` , `ease-in-out` , `cubic-bezier(...)` .
- **Délai** — attente avant de démarrer (optionnel).

2. Animations — keyframes

Pour des animations complexes, autonomes, en boucle :

```
@keyframes pulse {  
  0% { transform: scale(1); opacity: 1; }  
  50% { transform: scale(1.1); opacity: 0.8; }  
  100% { transform: scale(1); opacity: 1; }  
}  
  
.notification {  
  animation: pulse 1.5s ease-in-out infinite;  
}
```

Syntaxe `animation` combine 8 propriétés :

```
animation:  
  nom-keyframe  
  durée  
  fonction-timing  
  délai  
  nombre-itérations (ou `infinite`)  
  direction (normal | reverse | alternate)  
  remplissage (fill-mode: forwards | backwards | both)  
  état (paused | running);
```

3. Respecter `prefers-reduced-motion`

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    animation-duration: 0.01ms !important;  
    transition-duration: 0.01ms !important;  
  }  
}
```

→ Désactive les animations pour les utilisateurs qui ont ce réglage activé (vertiges, kinétose).

C. Méthodologie BEM

BEM = *Block, Element, Modifier*. Une convention de nommage pour des classes CSS **lisibles** et **sans collision**.

1. Le pattern

```
.bloc                → composant autonome  
.bloc__element       → partie d'un bloc  
.bloc--modificateur  → variante d'un bloc  
.bloc__element--modificateur → variante d'un élément
```

2. Exemple concret

```
<article class="card card--featured">  
  <header class="card__header">  
    <h2 class="card__title">Titre</h2>  
    <time class="card__date">23 avril 2026</time>  
  </header>  
  <div class="card__body">  
    <p class="card__text">Contenu</p>  
    <button class="card__btn card__btn--primary">Lire</button>  
  </div>  
</article>
```

```
.card { ... }  
.card--featured { border: 2px solid gold; }  
  
.card__header { ... }  
.card__title { font-size: 1.5rem; }  
.card__date { color: #999; }  
  
.card__btn { padding: 0.5rem 1rem; }  
.card__btn--primary { background: #007bff; }
```

3. Pourquoi BEM ?

- **Zéro collision** entre composants — `.card__title` ne peut pas entrer en conflit avec `.article__title`.
- **Lisibilité** — rien qu'en lisant le HTML, on comprend la structure du composant.
- **Spécificité faible** — tout est basé sur des classes (spécificité 10), facile à surcharger.
- **Pas besoin de sélecteurs imbriqués** — on cible directement avec une classe précise.

4. Autres conventions

BEM est la plus répandue, mais il existe :

- **SMACSS** — catégorise les styles (Base, Layout, Module, State, Theme).
- **OOCSS** — *Object-Oriented CSS*.
- **Utility-first** — Tailwind CSS, micro-classes (`.p-4` `.text-red-500`).

Choisissez-en une et **tenez-vous-y** dans un projet.

D. Performance CSS

1. Minification

Enlève espaces, commentaires, retours à la ligne :

```
/* Avant (150 octets) */
body {
  background-color: #fff;
  color: #333;
}
h1 {
  font-size: 2rem;
  margin-bottom: 10px;
}

/* Après (65 octets) */
body{background-color:#fff;color:#333}h1{font-size:2rem;margin-bottom:10px}
```

Outils : **CSSNano**, **Lightning CSS**, intégrés à Webpack/Vite automatiquement.

2. Sélecteurs simples vs complexes

```
/* ❌ Mauvais — 5 niveaux d'imbrication, spécificité 0,0,5 */
body .main .article .header .title { color: red; }

/* ✅ Bon — 1 classe, spécificité 0,1,0 */
.article-title { color: red; }
```

3. Animations GPU-friendly

Propriété animée	Coût	Type
width, height, top, left, margin	❌ Cher	Déclenche layout + paint
color, background, border	⚠️ Moyen	Déclenche paint
transform, opacity	✅ Cher	Compositing GPU seulement

Règle : pour animer, **préférer** **transform** (**translate**, **scale**, **rotate**) et **opacity**. Elles sont gérées par le GPU et ne provoquent pas de recalcul de mise en page.

```
/* ❌ Lag sur mobile */  
.box:hover { width: 300px; }  
  
/* ✅ Fluide à 60fps */  
.box:hover { transform: scaleX(1.5); }
```

4. Chargement critique

Au chargement initial, le CSS est **render-blocking** : la page reste blanche tant qu'il n'est pas chargé.

Techniques :

- **Extraire le critical CSS** (les ~10 premières kB nécessaires pour la partie visible) et l'inliner dans `<head>`.
- Charger le reste en asynchrone :

```
<link rel="preload" href="styles.css" as="style" onload="this.rel='stylesheet'">
```

- Utiliser des outils comme **Critters**, **critical** (npm) pour automatiser.

5. Checklist perf CSS

- ❌ CSS minifié en production.
- ❌ Fichier unique concaténé (ou HTTP/2 multiplexing, ok en plusieurs fichiers).
- ❌ Critical CSS inliné pour la partie visible.
- ❌ Animations via `transform` et `opacity` uniquement.
- ❌ Pas de sélecteurs profonds > 3 niveaux.
- ❌ `@font-face` avec `font-display: swap`.
- ❌ Ressources en cache avec `Cache-Control: max-age=31536000, immutable`.

E. Préprocesseurs et frameworks — panorama

1. Préprocesseurs (Sass / SCSS)

Sass ajoute au CSS : variables, imbrication, fonctions, mixins, boucles.

```
$primary: #007bff;

.btn {
  background: $primary;
  padding: 1rem;

  &:hover {
    background: darken($primary, 10%);
  }

  &--large {
    padding: 1.5rem;
  }
}
```

→ Compilé en CSS standard.

En 2026 : l'utilité a **diminué** grâce aux variables CSS natives + imbrication CSS native (venue récemment, supportée à 95 %). Sass reste utile pour les mixins et fonctions complexes, ou en environnement contraint.

2. Frameworks CSS

Framework	Approche	Cas d'usage
Bootstrap	Composants pré-stylisés + grille	Admin, dashboard, démarrage rapide
Tailwind CSS	Utility-first (classes atomiques)	Sites customs sans contrainte visuelle
Bulma	CSS pur, léger	Alternative à Bootstrap sans JS
Pure CSS	Minimaliste	Prototypes légers

Le cours Bootstrap (chapitre Bootstrap à J3) détaillera l'un d'entre eux.

3. Post-processeurs (PostCSS + Autoprefixer)

Autoprefixer ajoute automatiquement les préfixes navigateurs (`-webkit-` , `-moz-`) pour les propriétés récentes :

```
/* Vous écrivez */  
.carte { display: flex; }  
  
/* PostCSS + Autoprefixer produit */  
.carte {  
  display: -webkit-box;  
  display: -ms-flexbox;  
  display: flex;  
}
```

En 2026 : moins nécessaire qu'avant (les navigateurs modernes ne requièrent presque plus de préfixes), mais toujours une bonne pratique pour la compatibilité avec les vieux Safari / Chrome embarqué.

Conclusion

1. À retenir

- **Variables CSS** (`--nom`) natives, centralisent les valeurs, dynamiques en JS.
- **Transitions** pour animer un changement d'état, `@keyframes` pour des animations autonomes.
- **Animer** `transform` et `opacity` uniquement (GPU-friendly).
- **BEM** (`.bloc__element--modif`) : convention de nommage à zéro collision.
- **Sélecteurs courts** (1-2 classes max), spécificité basse.
- **Minification** + **critical CSS** inline = performance.
- **Respecter** `prefers-reduced-motion` et `prefers-color-scheme`.
- **Sass** est moins indispensable qu'avant ; **Tailwind** et **Bootstrap** restent utiles.

2. Pour aller plus loin

- **MDN — Custom Properties (variables CSS)**
- **MDN — CSS Animations**
- **BEM — Methodology**
- **web.dev — Rendering performance**
- **Sass** , **PostCSS**